Lecture Slip Questions: Answers

CSCI 235 Software Design and Analysis II, Hunter College
Lecture slip of Thursday, October 22, 2026 (Permutations and Operator Combinations)

Thank you for the questions on your slips. They are answered below. The table and the table of contents are links: click an entry to jump to its answer.

Your question	**Answer**
Can you post a thorough trace of slip questions 1 and 3?	Section 3, Section 4
When will the exit slips be graded?	Section 1
What does `printAll` do?	Section 2

Contents

1 When will the exit slips be graded? **2**

2 What does `printAll` do? **2**

3 Slip question 1: a thorough trace of `printAll("CAT", 0)` **3**

4 Slip question 3: a thorough trace of `ops` **4**

1 When will the exit slips be graded?

The exit slips are not graded. They are used only to check attendance. Nothing on a slip affects your grade, so a wrong or blank answer costs you nothing. The slips are for you: they show you what you can and cannot yet trace, and the questions you write on them are answered in these notes.

2 What does `printAll` do?

It prints every arrangement (permutation) of the letters of a string. For `"CAT"` it prints the $3! = 6$ orders: `CAT, CTA, ACT, ATC, TAC, TCA`.

```
void printAll(std::string& s, int k) {
  if (k == static_cast<int>(s.size())) { // every position filled
    std::cout << s << "\n";
    return;
  }
  for (int i = k; i < static_cast<int>(s.size()); ++i) {
    std::swap(s[k], s[i]);              // choose
    printAll(s, k + 1);                 // explore
    std::swap(s[k], s[i]);              // undo
  }
}
```

Analogy: filling seats in a row. Think of the positions `0, 1, 2, ...` as seats and the letters as people. `printAll(s, k)` means: *seats 0 to $k-1$ are already filled and will not change; now fill seat `k`.*

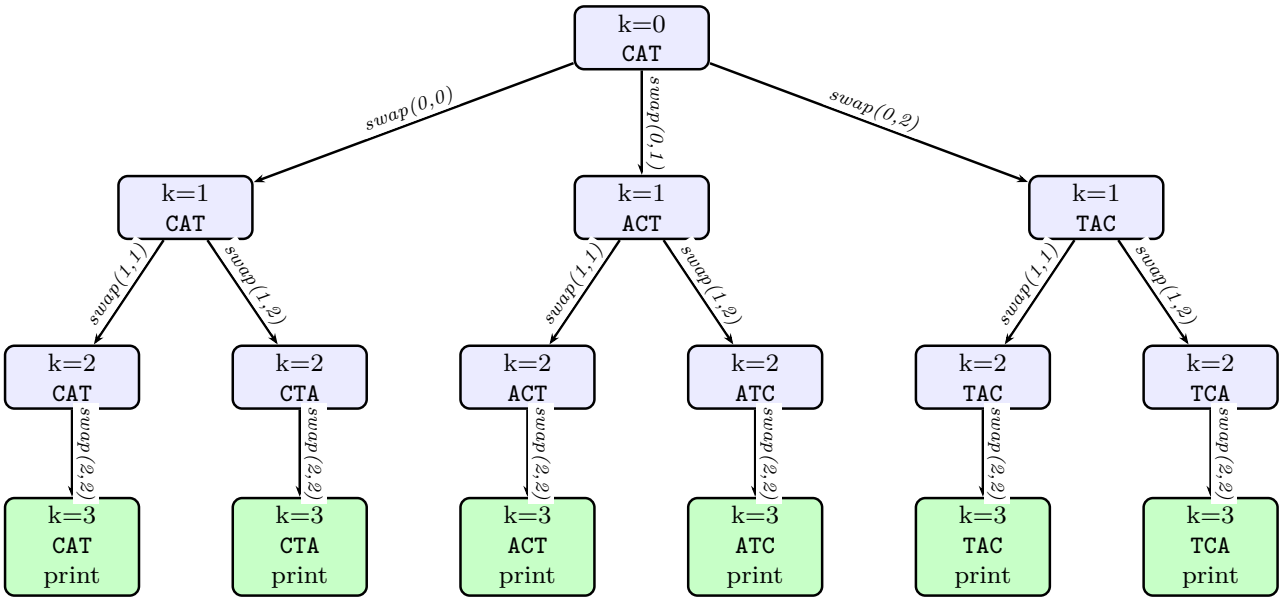
- **Who may sit in seat `k`?** Any person who has not been seated yet. Those people are in the positions `k` to the end of the string. The loop `for (int i = k; ...)` tries each of them in turn.
- **Choose.** `swap(s[k], s[i])` brings candidate `i` into seat `k`. (When `i == k`, the swap does nothing: the person already in seat `k` stays.)
- **Explore.** `printAll(s, k + 1)` fills the remaining seats, in every possible way.
- **Undo.** The second `swap` puts the string back exactly as it was, so the next candidate starts from the same arrangement.
- **Base case.** When `k == s.size()`, every seat is filled. The string `s` is one complete arrangement, so print it.

Why it counts $n!$. Seat 0 has n candidates, seat 1 has $n-1$, and so on down to 1: $n \times (n-1) \times \cdots \times 1 = n!$. This is the same "multiply the number of choices at each level" rule as in Class 14.

What stays true. At every call `printAll(s, k)`, the letters in positions $0, \ldots, k-1$ are fixed, and the letters in positions $k, \ldots, n-1$ are exactly the letters not yet used. Because of the undo, when a call returns, `s` is the same string it was when the call began.

3 Slip question 1: a thorough trace of `printAll("CAT", 0)`

Reading the diagram. Each box is one call of `printAll(s, k)`. It shows `k` and the string `s` *at the moment the call starts*. The label on an arrow is the swap that was done just before that call. Green boxes are calls with `k = 3`: they print. The calls run from left to right, depth first. There are $1 + 3 + 6 + 6 = 16$ calls.



Calls at each level. `k = 0`: 1 call. `k = 1`: 3 calls (one for each `i` at the top). `k = 2`: 6 calls (each `k = 1` call loops over `i = 1, 2`). `k = 3`: 6 calls, one per printed string. So the answer to part (c) of the slip is **3**.

Step by step, every swap and every undo. The indentation shows how deep the recursion is. The last column is the string `s` *after* the step.

#	Call	Event	s after
1	`printAll(s, 0)`	choose: swap(`s[0]`, `s[0]`) (no change) – call `printAll(s, 1)`	`CAT`
2	`  printAll(s, 1)`	choose: swap(`s[1]`, `s[1]`) (no change) – call `printAll(s, 2)`	`CAT`
3	`    printAll(s, 2)`	choose: swap(`s[2]`, `s[2]`) (no change) – call `printAll(s, 3)`	`CAT`
4	`      printAll(s, 3)`	**k = 3 = size: print**	`CAT` ← printed
5	`    printAll(s, 2)`	undo: swap(`s[2]`, `s[2]`) again	`CAT`
6	`  printAll(s, 1)`	undo: swap(`s[1]`, `s[1]`) again	`CAT`
7	`  printAll(s, 1)`	choose: swap(`s[1]`, `s[2]`) – call `printAll(s, 2)`	`CTA`
8	`    printAll(s, 2)`	choose: swap(`s[2]`, `s[2]`) (no change) – call `printAll(s, 3)`	`CTA`
9	`      printAll(s, 3)`	**k = 3 = size: print**	`CTA` ← printed
10	`    printAll(s, 2)`	undo: swap(`s[2]`, `s[2]`) again	`CTA`
11	`  printAll(s, 1)`	undo: swap(`s[1]`, `s[2]`) again	`CAT`
12	`printAll(s, 0)`	undo: swap(`s[0]`, `s[0]`) again	`CAT`
13	`printAll(s, 0)`	choose: swap(`s[0]`, `s[1]`) – call `printAll(s, 1)`	`ACT`
14	`  printAll(s, 1)`	choose: swap(`s[1]`, `s[1]`) (no change) – call `printAll(s, 2)`	`ACT`
15	`    printAll(s, 2)`	choose: swap(`s[2]`, `s[2]`) (no change) – call `printAll(s, 3)`	`ACT`
16	`      printAll(s, 3)`	**k = 3 = size: print**	`ACT` ← printed

#	Call	Event	s after
17	`printAll(s, 2)`	undo: swap(`s[2]`, `s[2]`) again	`ACT`
18	`printAll(s, 1)`	undo: swap(`s[1]`, `s[1]`) again	`ACT`
19	`printAll(s, 1)`	choose: swap(`s[1]`, `s[2]`) – call `printAll(s, 2)`	`ATC`
20	`printAll(s, 2)`	choose: swap(`s[2]`, `s[2]`) (no change) – call `printAll(s, 3)`	`ATC`
21	`printAll(s, 3)`	**k = 3 = size: print**	`ATC` ← printed
22	`printAll(s, 2)`	undo: swap(`s[2]`, `s[2]`) again	`ATC`
23	`printAll(s, 1)`	undo: swap(`s[1]`, `s[2]`) again	`ACT`
24	`printAll(s, 0)`	undo: swap(`s[0]`, `s[1]`) again	`CAT`
25	`printAll(s, 0)`	choose: swap(`s[0]`, `s[2]`) – call `printAll(s, 1)`	`TAC`
26	`printAll(s, 1)`	choose: swap(`s[1]`, `s[1]`) (no change) – call `printAll(s, 2)`	`TAC`
27	`printAll(s, 2)`	choose: swap(`s[2]`, `s[2]`) (no change) – call `printAll(s, 3)`	`TAC`
28	`printAll(s, 3)`	**k = 3 = size: print**	`TAC` ← printed
29	`printAll(s, 2)`	undo: swap(`s[2]`, `s[2]`) again	`TAC`
30	`printAll(s, 1)`	undo: swap(`s[1]`, `s[1]`) again	`TAC`
31	`printAll(s, 1)`	choose: swap(`s[1]`, `s[2]`) – call `printAll(s, 2)`	`TCA`
32	`printAll(s, 2)`	choose: swap(`s[2]`, `s[2]`) (no change) – call `printAll(s, 3)`	`TCA`
33	`printAll(s, 3)`	**k = 3 = size: print**	`TCA` ← printed
34	`printAll(s, 2)`	undo: swap(`s[2]`, `s[2]`) again	`TCA`
35	`printAll(s, 1)`	undo: swap(`s[1]`, `s[2]`) again	`TAC`
36	`printAll(s, 0)`	undo: swap(`s[0]`, `s[2]`) again	`CAT`

Check against the slip. (a) The printed strings, in order, are `CAT, CTA, ACT, ATC, TAC, TCA`. (b) After `printAll(s, 0)` returns, `s` is `"CAT"` again: the last row of the table undoes the last swap. Every choose is matched by an undo, so the rows come in pairs.

Watch the undo at work. Rows 11 and 12 undo the swaps that produced `CTA`, so `s` is `CAT` again. Only then does row 13 do `swap(s[0], s[1])`, which brings `A` to the front and gives `ACT`. If the undo in row 11 were missing, row 13 would swap letters of `CTA` instead, and the output would be wrong.

4 Slip question 3: a thorough trace of ops

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
  if (k == static_cast<int>(v.size())) { std::cout << expr << "\n"; return; }
  for (char op : std::string("+-*/")) {
    std::size_t len = expr.size();
    expr += op;
    expr += std::to_string(v[k]);      // choose
    ops(v, k + 1, expr);               // explore
    expr.resize(len);                  // undo
  }
}
// std::vector<int> v = {4, 5, 6, 7}; std::string e = "4"; ops(v, 1, e);
```

What it does. `expr` starts as `"4"`. At each position `k` the function appends one operator (tried in the order `+ - * /`) and the number `v[k]`, then recurses to fill the next gap. When `k == 4`, all three gaps are filled, so it prints the string. It never evaluates anything. The undo `expr.resize(len)` cuts the string back to the length it had before the operator and number were appended.

The shape of the search. There are 3 gaps (between `4,5`, `5,6`, `6,7`) and 4 choices for each.

Call	Gap being filled	Number of calls	Why
`ops(v, 1, expr)`	between 4 and 5	1	the first call
`ops(v, 2, expr)`	between 5 and 6	4	one per operator at gap 1
`ops(v, 3, expr)`	between 6 and 7	$4 \times 4 = 16$	one per operator at gaps 1, 2
`ops(v, 4, expr)`	none left: print	$4 \times 4 \times 4 = 64$	one per complete expression

So 64 lines are printed (part (a) of the slip), from $1 + 4 + 16 + 64 = 85$ calls. The *last* gap changes fastest, because the innermost loop runs the most often.

Step by step: the first 24 events. The undo row shows the string after `resize`. Notice that the string grows by two characters on each choose and shrinks by two on each undo.

#	Call	Event	expr after
1	`ops(v, 1, expr)`	choose: append `+` and 5; call `ops(v, 2, expr)`	`4+5`
2	`  ops(v, 2, expr)`	choose: append `+` and 6; call `ops(v, 3, expr)`	`4+5+6`
3	`    ops(v, 3, expr)`	choose: append `+` and 7; call `ops(v, 4, expr)`	`4+5+6+7`
4	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5+6+7` ← printed
5	`    ops(v, 3, expr)`	undo: `expr.resize(len)`	`4+5+6`
6	`    ops(v, 3, expr)`	choose: append `-` and 7; call `ops(v, 4, expr)`	`4+5+6-7`
7	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5+6-7` ← printed
8	`    ops(v, 3, expr)`	undo: `expr.resize(len)`	`4+5+6`
9	`    ops(v, 3, expr)`	choose: append `*` and 7; call `ops(v, 4, expr)`	`4+5+6*7`
10	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5+6*7` ← printed
11	`    ops(v, 3, expr)`	undo: `expr.resize(len)`	`4+5+6`
12	`    ops(v, 3, expr)`	choose: append `/` and 7; call `ops(v, 4, expr)`	`4+5+6/7`
13	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5+6/7` ← printed
14	`    ops(v, 3, expr)`	undo: `expr.resize(len)`	`4+5+6`
15	`  ops(v, 2, expr)`	undo: `expr.resize(len)`	`4+5`
16	`  ops(v, 2, expr)`	choose: append `-` and 6; call `ops(v, 3, expr)`	`4+5-6`
17	`    ops(v, 3, expr)`	choose: append `+` and 7; call `ops(v, 4, expr)`	`4+5-6+7`
18	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5-6+7` ← printed
19	`    ops(v, 3, expr)`	undo: `expr.resize(len)`	`4+5-6`
20	`    ops(v, 3, expr)`	choose: append `-` and 7; call `ops(v, 4, expr)`	`4+5-6-7`
21	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5-6-7` ← printed
22	`    ops(v, 3, expr)`	undo: `expr.resize(len)`	`4+5-6`
23	`    ops(v, 3, expr)`	choose: append `*` and 7; call `ops(v, 4, expr)`	`4+5-6*7`
24	`      ops(v, 4, expr)`	**k = 4 = size: print**	`4+5-6*7` ← printed

What happens next. Rows 1 to 15 finish the branch where gap 2 is `+`: four strings (rows 4, 7, 10, 13: `4+5+6+7`, `4+5+6-7`, `4+5+6*7`, `4+5+6/7`) printed one at a time, with undo rows in between, and row 15 returns the string to `4+5`. Row 16 starts the `-` branch at gap 2 and the same pattern repeats. The output comes in blocks of 4 lines that differ only in the last gap, and blocks of 16 lines that share the same first gap. These are the four columns of the appendix.

Part (b) of the slip. The count is $4! \times 4^3 \times 5 = 24 \times 64 \times 5 = 7\,680$. The three factors are independent: $4! = 24$ orders of the four numbers, $4^3 = 64$ ways to choose the three operators, and 5 ways to parenthesize four operands. Multiply them, because each choice can be made independently

of the others. Some of the 7 680 expressions have equal values (for example $(4 + 5) \times (6 + 7)$ and $(5 + 4) \times (6 + 7)$), because the count is of *expressions*, not of distinct values.

Appendix: all 64 lines printed by `ops(v, 1, e)`

first gap `+`	first gap `-`	first gap `*`	first gap `/`
1. `4+5+6+7`	17. `4-5+6+7`	33. `4*5+6+7`	49. `4/5+6+7`
2. `4+5+6-7`	18. `4-5+6-7`	34. `4*5+6-7`	50. `4/5+6-7`
3. `4+5+6*7`	19. `4-5+6*7`	35. `4*5+6*7`	51. `4/5+6*7`
4. `4+5+6/7`	20. `4-5+6/7`	36. `4*5+6/7`	52. `4/5+6/7`
5. `4+5-6+7`	21. `4-5-6+7`	37. `4*5-6+7`	53. `4/5-6+7`
6. `4+5-6-7`	22. `4-5-6-7`	38. `4*5-6-7`	54. `4/5-6-7`
7. `4+5-6*7`	23. `4-5-6*7`	39. `4*5-6*7`	55. `4/5-6*7`
8. `4+5-6/7`	24. `4-5-6/7`	40. `4*5-6/7`	56. `4/5-6/7`
9. `4+5*6+7`	25. `4-5*6+7`	41. `4*5*6+7`	57. `4/5*6+7`
10. `4+5*6-7`	26. `4-5*6-7`	42. `4*5*6-7`	58. `4/5*6-7`
11. `4+5*6*7`	27. `4-5*6*7`	43. `4*5*6*7`	59. `4/5*6*7`
12. `4+5*6/7`	28. `4-5*6/7`	44. `4*5*6/7`	60. `4/5*6/7`
13. `4+5/6+7`	29. `4-5/6+7`	45. `4*5/6+7`	61. `4/5/6+7`
14. `4+5/6-7`	30. `4-5/6-7`	46. `4*5/6-7`	62. `4/5/6-7`
15. `4+5/6*7`	31. `4-5/6*7`	47. `4*5/6*7`	63. `4/5/6*7`
16. `4+5/6/7`	32. `4-5/6/7`	48. `4*5/6/7`	64. `4/5/6/7`